COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Module 27: Animation, audio and interactive timing

Four expanded lessons • Project and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Module 27: Animation, audio and interactive timing 4

Lesson 27.1: Skeletons, transforms and skinning . . . 5

Lesson 27.2: Animation blending, state machines and inverse kinematics . . . 8

Lesson 27.3: Digital audio, mixing and real-time callback discipline . . . 11

Lesson 27.4: Spatial audio, animation events and multimodal accessibility . . . 14

Module 27 project . . . 16

Module 27 review . . . 17

Reference Directory 18

S01: Collision documentation . . . 18

S02: Simulation documentation . . . 18

S03: glTF tutorial: Skins . . . 18

S04: glTF 2.0 specification . . . 18

S05: Writing a callback function . . . 18

S06: SDL_AudioStreamCallback . . . 18

S07: Xbox Accessibility Guidelines . . . 18

S08: CS188: Informed search . . . 19

S09: Introduction . . . 19

S10: CS188: Minimax . . . 19

S11: Automatic differentiation with torch.autograd . . . 19

S12: Networked movement in the Character Movement Component . . . 19

S13: Snapshot Interpolation . . . 19

S14: GGPO SDK repository 19

S15: Transaction documentation 20

S16: Atomic Commit in SQLite 20

S17: os.replace documentation 20

S18: zipfile documentation 20

S19: Fix Your Timestep! 20

S23: XAG 103: Additional channels for audio and visual cues 20

S24: Deterministic Lockstep 20

S25: XAG 107: Input 20

S26: XAG 101: Text display 21

Original program reading IDs 21

Module 27: Animation, audio and interactive timing

Prerequisites: Modules 07, 10, 18 and 24.

Original reading IDs: R19, R20, R22, R26. See the reference directory.

Module goal: Integrate an animated character or articulated object with event-driven audio. Demonstrate smooth interpolation, bounded audio work, adjustable sound categories and correct behavior when animation rate differs from simulation rate.

Lesson 27.1: Skeletons, transforms and skinning

Learning goal: Compute hierarchical animation without mixing coordinate spaces.

Coordinate spaces are part of the asset format

An articulated object is a hierarchy, not a bag of independent rotations. Choose column-vector notation for this lesson. A local joint transform maps from the joint's coordinates to its parent's coordinates. The world transform is parent_world * local. A root uses the skeleton's world transform as its parent. Multiplication order is a semantic contract, not a stylistic preference.

Evaluate ancestors before descendants. Imported node order is not necessarily parent-first, so either topologically sort the hierarchy or evaluate recursively with cycle detection and a depth bound. An invalid parent index, a cycle or an excessive depth is malformed asset data. Treat it as a load error rather than letting animation recurse until the stack fails.

Derive skinning from the bind pose

Let B_j be joint j's world transform in the bind pose and G_j its current world transform, all expressed in compatible spaces. To move a bind-space vertex through joint j, first map it into the joint's bind coordinates with inverse(B_j), then move it into the current pose with G_j. The resulting transform is G_j * inverse(B_j). If the renderer expects positions in mesh-local space, include inverse(mesh_world) on the left. Khronos's skinning tutorial and glTF specification make the joint and inverse-bind relationships explicit [S03, S04].

Linear blend skinning forms a weighted sum of transformed positions:

```
p_world = sum_j weight_j * G_j * inverse(B_j) * p_bind
sum_j weight_j = 1
```

In the bind pose, G_j=B_j, so each product is identity and the vertex reproduces its original position. This identity test is more valuable than immediately importing a detailed character. Validate finite, nonnegative weights, valid joint indices and a normalization policy. A zero total weight needs an explicit fallback or rejection; division by zero is not an acceptable repair.

Worked two-joint arm

Joint 0 is at the origin. Joint 1's local translation is (1,0), so its bind world position is (1,0). A vertex bound entirely to joint 1 lies at (2,0) in bind world coordinates. Its joint-local bind position is (1,0). Rotate joint 0 by 90 degrees while preserving joint 1's local translation. Joint 1 moves to (0,1), and the vertex moves to (0,2). Translating each joint independently without inheriting the parent rotation gives the wrong result.

For a blended vertex, use two weights of 0.5. At bind pose, both corrected transforms are identity, so blending still preserves the original vertex. Omitting inverse-bind correction adds the joint's bind placement again and produces displacement even before animation starts. This is a coordinate-space defect, not a bad blend curve.

Orientation and deformation

Interpolate translation and scale separately from rotation. A quaternion and its negative represent the same orientation; negate one endpoint when their dot product is negative before a shortest-path interpolation. Normalized linear interpolation is useful for small angular differences, while spherical interpolation better preserves constant angular speed between two orientations. Near-equal quaternions need a numerically stable branch. An unnormalized quaternion can introduce invalid transforms.

Linear blend skinning can collapse volume near twists. This is a known model limitation, not always a shader bug. For normals, the transform depends on the skinning and scaling model; nonuniform scale cannot be handled by blindly reusing a position transform. In the core lab, restrict joints to rigid transforms and document that restriction before adding more advanced deformation. Correctness within a declared model is preferable to silently accepting unsupported assets.

Asset and runtime ownership

Store the immutable hierarchy, bind transforms and clips as resources. Each character owns mutable playback time, blend state and evaluated pose buffers. Sharing a mutable pose across characters can make one actor's animation overwrite another's. The renderer receives a snapshot of joint matrices or a clearly synchronized buffer; it must not read a pose while a job is half finished.

The Python kernel uses 2D transforms to make the parent composition visible. The full lab adds a small skinned mesh or line skeleton to the engine, checks the bind identity, and displays local axes. Test a parent rotation, a translated mesh root, one invalid joint index and a rejected hierarchy cycle. Render at different rates without changing the authoritative animation time. Explain which transforms are local, skeleton-relative, mesh-relative and world-relative in the submission.

Original worked checkpoint

A vertex influenced equally by two joints has weights 0.5 and 0.5. Omitting the bind-pose correction can move it even when the skeleton is supposedly in its original pose.

Full practical assignment

Animate a two-joint mesh or line skeleton. Pass: the bind pose reproduces the source geometry, weights are validated and a parent rotation moves descendants consistently.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Parent-first two-dimensional pose evaluation**. Run `starter/python/demos/lesson_27_1.py`; implement `starter/python/exercises/lesson_27_1.py`.

Assigned study and native transfer

R19 : transformations and animation; R26 : skeletal animation concepts.

Read S03 in full, then S04 sections on skins and animations. Implement the two-joint identity test before loading complex art.

Engine transfer: Add an immutable skeleton resource and a per-instance pose buffer to the existing engine. Demonstrate two actors sharing one skeleton without sharing playback state.

Check your understanding

1. Why evaluate parent transforms before dependent children?
2. What should ordinary normalized skinning weights sum to?
3. Why avoid directly interpolating arbitrary rotation-matrix entries?
4. A root rotates 90 degrees and its child has local translation (1,0). Where is the child in world coordinates under column-vector parent*local composition?
5. Why should G_j * inverse(B_j) be identity in the bind pose, and what extra transform is needed for a mesh-local output buffer?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.2: Animation blending, state machines and inverse kinematics

Learning goal: Coordinate authored motion with runtime goals.

Animation is a controlled state transition

A character does not simply play whichever clip was requested last. Define an animation state machine with entry behavior, exit behavior, interruption rules and transition duration. Idle, walk and action states are enough for this lab. Specify whether an action can be interrupted by movement, damage or pause, and whether a repeated trigger restarts it or is ignored.

Separate state selection from pose evaluation. A transition blends sampled poses, but the gameplay action may have an exact authoritative start tick. The blend's visual midpoint is not automatically the moment a hit occurs. Blend translation and rotation using the conventions from Lesson 27.1. A linear blend parameter rises from 0 to 1 over a declared duration; a smooth curve changes derivative continuity, not the underlying event ownership.

Phase and root motion

Two walk clips can have compatible speeds but incompatible foot phases. Blending left-foot contact with right-foot contact can produce sliding. Normalize cycle phase or use markers when the asset design requires it. Clip time must have a unit: seconds, normalized cycle fraction or ticks. A variable called time without a unit is a common source of integration mistakes.

Root motion is displacement authored by animation. Physics-driven movement instead derives displacement from simulation. Choose one authority. A useful contract is that animation proposes a displacement, the movement controller resolves collision and accepts a corrected displacement, and the visual pose adapts to the result. Applying both the full authored displacement and the physics velocity double-counts movement. Teleporting a dynamic body to every animation root sample bypasses its physical constraints.

Derive a two-link inverse-kinematics solution

For a planar arm with lengths L1 and L2 and target (x,y), let d=sqrt(xx+yy). The reachable radial range without joint-angle limits is abs(L1-L2) <= d <= L1+L2. The law of cosines gives an elbow angle theta2 through:

```
c2 = (d*d - L1*L1 - L2*L2) / (2*L1*L2)
theta2 = acos(clamp(c2, -1, 1))
theta1 = atan2(y,x)
         - atan2(L2*sin(theta2), L1+L2*cos(theta2))
```

The negative theta2 branch gives the alternate elbow configuration. A pole direction, previous pose or joint-limit policy selects a branch. Clamp unreachable targets to the reach interval and report that clamping occurred. The clamp on c2 handles small rounding excursions, but it must not conceal an invalid negative link length.

Worked target and singularity

With L1=L2=1 and target (1,1), d squared is 2, c2=0, theta2=90 degrees and theta1=0 degrees for the chosen branch. Forward kinematics returns (1,1). For target (3,0), the maximum reach is 2, so the arm can extend toward (2,0) but cannot reach (3,0). Reporting a residual of 1 is honest; repeatedly iterating toward an impossible target is wasted computation.

At the origin with equal links, a folded arm can reach the target but the shoulder direction is not uniquely determined. At full extension, small target changes can create large angular changes. Preserve a stable prior direction or use an explicit fallback. In 3D, singularities, joint limits and multiple chains add complexity; this analytic 2D solver is a teaching model rather than a general full-body IK system.

Ordering the animation pipeline

A typical bounded pipeline samples base clips, blends them, applies an additive layer, solves selected IK constraints, then computes final world transforms. The exact ordering matters. Solving foot IK before an unrelated later layer moves the pelvis can undo the contact. Recompute dependent transforms after local changes. Cache immutable clip data, not partially evaluated mutable poses shared across actors.

Trace each transition with source state, target state, trigger, start tick, blend duration and interruption reason. When a character freezes, the trace should reveal whether a state never exited, a clip time stopped or an IK target became invalid. Do not debug all three through a single final mesh screenshot.

Deliberate practice

The exercise kernel computes the two-link solution and reports reachability. The full lab must also implement idle/walk/action transitions, cancellation and root-motion ownership. Test target motion across the reach boundary, a pause during a blend, a movement trigger during an action and two consecutive interruptions. Verify pose continuity numerically and inspect behavior at multiple rendering rates. A correct endpoint equation alone does not complete an animation controller.

Original worked checkpoint

A two-link arm with lengths 1 and 1 cannot reach a target 3 units from its base. A solver should clamp or report the unreachable condition rather than generating invalid angles.

Full practical assignment

Build idle/walk/action transitions and a two-link IK demonstration. Pass: interruptions behave as specified, unreachable targets are handled, and root-motion ownership is documented.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Analytic two-link inverse kinematics**. Run `starter/python/demos/lesson_27_2.py`; implement `starter/python/exercises/lesson_27_2.py`.

Assigned study and native transfer

R19 : animation and inverse kinematics; R26 : animation blueprints and blending concepts, compared with your own small implementation.

Read S04 animation interpolation and node transforms. Revisit R19 animation and R26 animation-system concepts. Derive the two-link geometry from the law of cosines.

Engine transfer: Implement the state/pose separation in your engine and make the animation trace visible in the inspector. Add one cancellation test that frees an action-owned resource.

Check your understanding

1. Can every IK target be reached?
2. What is the maximum reach of two collinear links of length 1 each?
3. Why define root-motion ownership?
4. For links of length 1 and 1 and target (1,1), compute one elbow and shoulder solution. Verify it with forward kinematics.
5. An action clip moves the root 0.3 m while the physics controller also moves it 0.3 m in the same tick. What contract is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.3: Digital audio, mixing and real-time callback discipline

Learning goal: Generate sound without missing the output deadline.

Audio runs on a different deadline

A sample is one channel's value at one instant. An audio frame contains one sample for every channel. At 48,000 frames per second, 256 stereo frames contain 512 sample values but span only 256/48000 = 0.005333 seconds. Confusing samples, frames and bytes can halve a buffer or overrun it. State the format: sample rate, channel count, sample type and interleaving.

A game may tolerate an occasional long visual frame; the audio device still needs its next buffer. Disk access, memory allocation, waiting on a mutex, console logging and unpredictable callbacks can exceed the available time. PortAudio's callback guidance explicitly cautions against blocking and unpredictable work [S05]. A callback's practical deadline also depends on the platform and buffering arrangement; the buffer duration is a useful budget calculation, not proof of end-to-end latency.

Generate and mix a signal

A sine oscillator advances phase by 2*pi*frequency/sample_rate each frame. Preserve phase between callbacks or each buffer boundary can introduce a discontinuity. For a mono mix, y[n] = sum_i gain_i*x_i[n]. A conservative no-clipping policy for sources bounded in [-1,1] is sum_i abs(gain_i) <= 1. Two voices at gains 0.25 each have a worst-case peak of 0.5, even when their phases align.

A limiter or soft clipper is a different policy and changes the waveform. Do not call saturation distortion-free. Abrupt gain or playback changes can create clicks; use a short ramp with a specified duration. Choose frequencies below the Nyquist limit, but remember that discontinuous or nonlinear waveforms can introduce harmonics above it. The core lab uses generated tones so no third-party assets or codecs are needed.

Bounded work and ownership

Keep a fixed-size voice array. A callback processes at most a declared number of control commands and at most the configured voice count times the output frame count. Even a lock-free queue can be abused by draining until empty while the producer keeps filling it. Bound commands per callback. If the queue fills, drop or coalesce noncritical updates under an explicit policy and count the event outside the callback.

A single-producer/single-consumer ring has one writer for its write index and one writer for its read index. The producer writes a slot before publishing the new write index with release ordering. The consumer observes that index with acquire ordering before reading the slot. Slot reuse requires the reverse publication of the read index. This ownership pattern does not extend automatically to multiple producers. The native C++ companion makes these boundaries concrete and checks whether its atomic index type is lock-free on the build target.

Allocate and decode before playback. Pass stable handles or immutable sample buffers to audio, not pointers to resources that the game thread may immediately free. A stop command must be acknowledged or otherwise synchronized before resource reclamation. An audio callback should not depend on a renderer lock or gameplay allocation to finish.

Worked budget and stress test

Suppose the chosen device consumes 256-frame buffers at 48 kHz. An illustrative measured callback cost of 0.7 ms leaves about 4.63 ms relative to the buffer duration. That is not a guarantee against scheduling delays. Record maximum observed callback time, distribution, underrun count if exposed, voice count and the measurement method. Never invent an underrun count when the API does not expose one; report the available observation and its limitation.

The native harness times offline callback invocations, so it can test arithmetic, bounded command processing and queue behavior. It does not prove that a real device is underrun-free. The full lesson requires the learner to connect a maintained platform API, mix two tones, vary gains and run a deadline-stress test. SDL3 stream callbacks have their own thread and API contracts; consult the selected SDL3 page rather than copying SDL2 callback code [S06].

Completion boundary

The Python exercise creates offline samples to verify phase continuity and gain bounds. Python execution is not presented as a real-time audio callback implementation. The C++ harness demonstrates a native ownership pattern without linking a device library. Actual hardware playback, mute/device-loss behavior and callback stress evidence remain required lab work. Start at low output volume and avoid testing with unexpectedly loud peaks. Preserve separate music/effects settings and a master mute for the next lesson.

Original worked checkpoint

A 256-frame buffer at 48,000 frames per second spans 256/48,000 seconds, about 5.33 ms. A callback that blocks for 20 ms can miss that timing requirement.

Full practical assignment

Mix two generated tones with adjustable gains using a platform audio API. Pass: no clipping under the documented gain policy, bounded callback work and a recorded underrun or deadline-stress test.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Offline phase-continuous tone mixer**. Run `starter/python/demos/lesson_27_3.py`; implement `starter/python/exercises/lesson_27_3.py`.

Assigned study and native transfer

R10 from Volume 2: sampling and filtering; R22 : SDL3 audio; R20 : audio systems.

Read S05 callback guidance and S06 SDL3 stream-callback contract. Revisit Module 18 for memory ordering. Record exact device/API format and version in your lab report.

Engine transfer: Connect the native mixer pattern to your engine audio adapter. Keep device-specific code outside the simulation and demonstrate graceful operation when no output device exists.

Check your understanding

1. What duration does 256 frames represent at 48 kHz?
2. Why avoid disk reads inside an audio callback?
3. Does halving the buffer automatically improve every aspect of audio?
4. How many sample values are in 256 stereo frames, and what duration do they represent at 48 kHz?
5. Why can draining a lock-free command queue until empty still violate a callback deadline? Give a bounded alternative.

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.4: Spatial audio, animation events and multimodal accessibility

Learning goal: Synchronize feedback while preserving user control.

A game event is not an audio call

The simulation decides that a relay activated. Presentation decides how that fact is conveyed: a tone, an on-screen message, an animation or a vibration where supported. Keep the authoritative event separate from any one output channel. Otherwise muting sound may accidentally disable gameplay logic, or replaying physics may trigger a second reward.

Use an event identity such as (match_epoch, tick, entity_id, local_event_index). The identity must be stable when the same deterministic tick is replayed. A pointer or wall-clock timestamp is unsuitable. Store presentation deduplication separately from rollback state so restoring a simulation snapshot does not erase the memory of already-played cues. Bound the deduplication window by the rollback horizon plus a safety margin, and reset it when the match epoch changes.

Event policy under prediction

Some predicted effects can happen immediately and be corrected later; others should wait for confirmation. A footstep can be deduplicated with low consequence. A purchased reward must be durably confirmed and cannot be treated as an audio-style best effort. If a predicted cue turns out not to have happened, the application cannot unplay the sound. This is a policy tradeoff, not an algorithm that eliminates all perceptual error.

Animation markers need similar identity. If a clip crosses a marker while the renderer skips a frame, the marker should still fire based on the traversed time interval. If playback loops, include a cycle index. Pausing and seeking require explicit marker semantics. Firing a marker whenever the current time is greater than its threshold causes it to fire every subsequent frame.

Derive a simple spatial pan

Stereo panning is a teaching approximation, not a full head-related transfer function. Let p range from -1 for left to +1 for right. Equal-power gains can be defined by angle=(p+1)*pi/4, left=cos(angle), right=sin(angle). The sum of squared gains is one. At center, each gain is about 0.7071, not 0.5. This preserves the mathematical power sum for an isolated signal; perceived loudness still depends on content and playback conditions.

Distance attenuation is another independent mapping. Choose a bounded rule, such as gain=1/(1+(distance/reference_distance)^2), and state that it is a game-design curve rather than a complete acoustic model. An occlusion test can drive filtering, but avoid running expensive geometry queries inside the audio callback. The game thread sends a bounded parameter update instead.

Worked multi-clock trace

A simulation runs at 60 Hz. Tick 90 spans authoritative time 1.5 to about 1.5167 s. A foot-contact marker generates event (epoch7,90,actor12,0). The display renders twice before the next simulation step, but both frames observe the same event identity, so it is presented once. A later correction restores tick 88 and replays through tick 90. The event is regenerated with the same identity and is not presented again.

If the restored simulation produces a different contact at tick 90, its deterministic local index and event type must follow the schema. Never deduplicate solely by tick: multiple legitimate events can occur in one tick. Never allow an unbounded global set to retain every event from a multi-hour session.

Accessibility is part of the event contract

A critical audio cue needs another usable channel. For the relay game, activation can produce a visible status change and readable text in addition to a tone. Color alone should not carry the entire distinction; use a shape, icon or label as well. Music, effects, speech and master volume have different user purposes. Muting one category must not suppress a necessary visual cue. Microsoft XAG 103 addresses additional channels for critical information [S23].

Test the complete task with output muted, without relying on a tester remembering where the cue would have been. Test whether captions describe important non-speech events, whether text persists long enough and whether simultaneous cues remain understandable. A guideline review helps identify barriers but does not replace user evaluation or establish legal compliance [S07].

Deliberate practice

Implement event deduplication and the equal-power pan kernel. Then connect events to separate audio and visual adapters. Record a replay trace containing a repeated event, a second legitimate event at the same tick and a new match epoch. The full lab adds separate category controls, a bounded event cache and a muted-audio task test. Do not let the success of a deduplication unit test stand in for actual accessible presentation.

Original worked checkpoint

A predicted footstep may be encountered again during replay. Assigning an event identity and a presentation policy prevents an unintended duplicate sound while preserving the corrected simulation.

Full practical assignment

Add spatial panning, separate music/effects controls and a visual equivalent for a critical audio cue. Pass: replay does not duplicate one-shot events and a muted-audio test still allows the required task.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Equal-power pan and stable presentation events**. Run `starter/python/demos/lesson_27_4.py`; implement

`starter/python/exercises/lesson_27_4.py`.

Assigned study and native transfer

R20 : audio and event systems; R22 : audio interfaces; R29 from the resource directory: audio and subtitle accessibility.

Read S23 additional channels and S07 guideline scope. Revisit S14 for rollback context. Review the original R22/R26 event and audio integration material.

Engine transfer: Route your engine events into interchangeable audio, visual and optional haptic adapters. Demonstrate one critical task with audio muted and no dependence on color alone.

Check your understanding

1. Is stereo panning a complete spatial-hearing model?
2. Why give presentation events stable identities during replay?
3. Why provide a non-audio equivalent for a critical cue?
4. At center pan under the equal-power rule, what are the left and right gains? What quantity is preserved?
5. A rollback snapshot restores the set of already-played sounds to an earlier version. What bug follows, and where should this set live?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 27 project

Build an Articulated Signal Station. A two-link arm or small character can idle, move and perform one interrupted action. Its motion must have a defined hierarchy, bind pose and blend policy. One semantic gameplay event drives an audio cue and a visual alternative. The example theme is optional; an original equivalent is welcome.

Prepare the native audio resources outside the callback, route bounded commands through an ownership-safe channel and keep music/effects controls independent. Implement the actual platform audio path and record a stress test. The supplied offline harness establishes only the behavior it executes; it does not establish the student’s device deadline or the absence of dropouts.

Run the simulation at a fixed step and show presentation at two rates. Roll back across an already presented event and demonstrate that a one-shot is not duplicated. Complete a muted-audio user task and record where the tester hesitated. Do not claim broad accessibility from a single participant.

Deliver transform/space definitions, action state table, IK tests, callback resource/work caps, raw device observations, event identity/retention policy, settings save/load and the

muted-task record. The reviewer should be able to trigger cancellation and identify which system owns motion, sound emission and cleanup.

Module 27 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A bind-pose mesh moves when skinning is enabled. Give a structured diagnosis.
2. An action is canceled during a blend but its attack sound still fires. What ownership rules are missing?
3. A callback has a lock-free queue but misses deadlines. Give three plausible reasons.
4. Why should the presentation-event deduplication ledger not be restored with the rolled-back simulation state?
5. What makes a critical visual audio alternative successful rather than merely present?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source